Introduction to Object-Oriented Programming (OOP)

Practical Session Guide

1 Part 1: Theoretical Background (A Simple Introduction)

1.1 1. What is Object-Oriented Programming (OOP)?

In your first year, you were accustomed to **Procedural Programming** (like the C language), where the focus was on **Functions** and a sequence of commands to solve a problem.

Object-Oriented Programming (OOP) is a different mindset that closely mimics how we view the real world. Instead of focusing on functions, we focus on **Objects**.

1.2 2. The Core Concepts (Class vs. Object)

- **Class:** This is the template or blueprint. It describes the attributes and behaviors that an object will have.
- **Object:** This is the real, tangible instance created from that blueprint.
- *Example:* A `Class` is the architectural blueprint for a car, while an `Object` is your actual car with its specific color and speed.

1.3 3. Class Structure in Java

Any class consists of two main parts:

1. **Attributes (Fields):** The variables that hold the state (e.g., student name).
2. **Methods (Behaviors):** The functions that define the actions (e.g., a function to print information).

2 Part 2: Guided Practical Work (Guided TP) — Eclipse Environment

2.1 Step 1: Create the Project

1. Open **Eclipse IDE** and select your Workspace.
2. Go to `File > New > Java Project`, name it `TP1_IntroOOP`, and click **Finish**.

2.2 Step 2: Program the Blueprint (Class `Student`)

1. Inside the `src` folder, create a new class named `Student`.
2. Write the following code:

Listing 1: Student.java

```java

public class Student {
    // Attributes (الخصائص)
    String name;
    int id;
    String major;

    // Methods (السلوكيات)
    public void displayInfo() {
        System.out.println("Student Name: " + name);
        System.out.println("Student ID: " + id);
        System.out.println("Major: " + major);
        System.out.println("--------------------------");
    }
}
```

2.3 Step 3: The Main Program (Main Class)

1. Create a new class named `MainApp` and check the option to generate the `public static void main` method.
2. Write the following code to instantiate the objects:

Listing 2: MainApp.java

```java
public class MainApp {
    public static void main(String[] args) {
        // إنشاء الكائن الأول
        Student student1 = new Student();
        student1.name = "Ahmed";
        student1.id = 2026001;
        student1.major = "Computer Engineering";

        // إنشاء الكائن الثاني
        Student student2 = new Student();
        student2.name = "Sara";
        student2.id = 2026002;
        student2.major = "Cybersecurity";

        // استدعاء دالة الطباعة
        student1.displayInfo();
        student2.displayInfo();
    }
}
```

3. Click the **Run** button to verify that the program works correctly.

3 Part 3: Individual Tasks

Before pushing your project, complete the following tasks individually within the same project to solidify your understanding.

Task 1 (Add a New Attribute)

- Open the `Student` class. Add a new attribute of type `String` named `email`.

- Do not forget to update the `displayInfo()` method so it prints the email along with the rest of the student's information.

Task 2 (Instantiate a New Object)

- In the `MainApp` class, instantiate a third object named `student3`.
- Make this object represent **you**! (Enter your real name, university ID, major, and email), then call your `displayInfo()` method.

Task 3 (Mutating State)

- Things change in the real world, and so do objects in memory.
- In the `MainApp` class, at the end of your code (after printing `student1`'s information), write a line of code that changes `student1`'s major from `"Computer Engineering"` to `"Artificial Intelligence"`.
- Call `student1.displayInfo()` again to verify that the value has changed.

Task 4 (Add a New Behavior)

- Go back to the `Student` class. Add a new method named `study()` that returns nothing (`void`) and prints a simple message like: `"The student [name] is currently studying OOP."`
- Call this method for `student2` in the main program.

4 Part 4: Submission Assignment (Integration with GitHub)

Once you have completed the tasks above and verified that the program runs without errors, it is time to submit your practical work via GitHub:

1. Open **GitHub Desktop**.
2. Go to `File > Add local repository`, and select your `TP1_IntroOOP` project folder from your Workspace.
3. When the "create a repository" warning appears, click the link and ensure you select **Java** from the **Git ignore** dropdown menu (this is crucial to prevent uploading compiled files).
4. In the **Summary** field, write a clear commit message like: `Complete TP1 with all 4 tasks`.
5. Click the blue **Commit** button.
6. Publish the repository as a **Private Repository** and click **Publish**.
7. Log in to your GitHub account on the web, navigate to `Settings > Collaborators`, and add your professor's GitHub username `SniperDZ-Pro` to grant them access to evaluate your code.